



SMART CONTRACT SECURITY REVIEW

**Internal pre-audit review of the
CHERR.IO contracts**

VERSION 1.1 · OCTOBER 2026

CONTENTS

01	Summary	4
02	Scope and method	6
03	System overview	8
04	Severity definitions	10
05	Findings	11
06	Static analysis (Slither)	19
07	Testing and coverage	20
08	Properties checked	22
09	Centralisation and trust assumptions	23
10	Mainnet readiness checklist	24
11	Reproducing the results	25
00	Appendix A — Access control by function	26
00	Appendix B — Document history	28

ABOUT THIS REPORT

What this is. This report is a security review of the five Solidity contracts that hold and move donors' money on CHERR.IO: `Campaign`, `CampaignFactory`, `EmergencyPool`, `IEmergencyPool` and `PlatformConfig`. Version 1.0 reviewed source commit `a70d53d`. Version 1.1 (this one) re-reviews the contracts after the fix batch Data Vallis approved on 10 October 2026 (TASK-023c, ADR-061). The review combines four methods: a line-by-line manual review, static analysis with Slither, the project's Foundry suite (unit, fuzz and invariant tests, 261 tests in total), and tests written for this review that pin down each finding in executable form.

What this is not. This is an **internal, AI-assisted review**. It was carried out by Claude, the project's AI implementer and CTO, on behalf of Data Vallis d.o.o. It is **not an independent third-party audit** and does not replace one. CHERR.IO's own rule (Architecture §6) requires an external audit by a recognised firm before any mainnet deployment holding real funds. This report exists to make that audit faster and cheaper: the scope, the trust model, the known limitations and the open design questions are written down in advance, with evidence.

VERDICT IN ONE PARAGRAPH

No critical or high-severity issue was found. Donor funds cannot be taken by any role: campaign money can only go to the campaign's fixed beneficiary, back to donors, or to the Emergency Pool. Pool money can only go to factory campaigns. The two medium findings are **trust-concentration** issues, not code bugs. M-01 (one multisig holds every operational role) is fixed in the mainnet deployment script, which now requires three different Safes. M-02 (the 48-hour timelock can redirect future flows to the pool and the treasury) stays as a monitored, documented power (§9). Of the six low and ten informational findings, five were fixed in code in version 1.1: L-01, L-02, L-03, L-06 and I-01, plus the I-07 cap. Each fix is pinned by a test. The rest are acknowledged or accepted, with reasons.

01 SUMMARY

FINDINGS BY SEVERITY

SEVERITY	COUNT	FIXED IN CODE	OPEN — GOVERNANCE OR DESIGN DECISION
Critical	0	–	–
High	0	–	–
Medium	2	1 (M-01, deployment script)	1 (M-02, monitoring)
Low	6	4 (L-01, L-02, L-03, L-06)	2 acknowledged (L-04, L-05)
Informational	10	2 (I-01, I-07)	8 triaged, accepted or acknowledged

No finding required an emergency code change. Every contract change means a redeploy, because the contracts are deliberately non-upgradeable. For that reason all code changes went in **one batch** (TASK-023c, ADR-061): L-01, L-02, L-03 (fee per tranche), L-06, I-01 and I-07, plus three Safes in the mainnet deployment script. The batch is merged in the repository. **The Amoy test contracts still run version 1.0 code until Data Vallis redeploys them**; mainnet will deploy the fixed code.

WHAT IS DONE WELL

- **No upgradeability, no admin withdrawal.** There is no proxy and no `selfdestruct`. No function lets an owner move escrowed USDC to an address of their choosing.
- **Snapshotted rules.** Fee, success threshold, vote window, quorum, approval and release delay are copied into each campaign when it is created. Each Emergency Pool proposal copies quorum and approval the same way. A later configuration change never alters a running campaign's rules.
- **Pull-based refunds and checks-effects-interactions.** State is written before the token transfer. `nonReentrant` guards every function that moves tokens.
- **Conservation proven by invariants.** Over 32,768 random calls per invariant, a campaign's USDC balance always equals what it still owes. Raised never exceeds the target, and payouts are exact for both payout modes. The pool's balance equals its free balance plus its reserved allocations.
- **Bounded parameters.** The fee is capped at 5 %, the approval at no less than 50.01 %, the vote window at 1 hour–14 days and the release delay at 7 days or less. The refund window is 30–365 days.
- **Weighted, checkpointed voting.** Pool votes use the contributions recorded before the proposal block. Contributions are irreversible gifts, so flash-loan voting is impossible.

MOST IMPORTANT RECOMMENDATIONS BEFORE MAINNET

- 1** Commission the **external audit**, giving the auditor this report and its tests as the starting point.
- 2** **Create the three Safes** with different signer sets (M-01). The deployment script now refuses a shared Safe; choosing the people is up to Data Vallis.
- 3** **Monitor the timelock.** Alert on every `CallScheduled` that touches `setEmergencyPool`, `setTreasury` or role grants (M-02), and announce it publicly within the 48-hour window.
- 4** **Redeploy on Amoy** and re-run the end-to-end flows on the fixed contracts before the mainnet deployment.

02 SCOPE AND METHOD

IN SCOPE

FILE	LINES OF CODE (NON-COMMENT)	SHA-256 (FIRST 16 HEX)	ROLE
<code>src/Campaign.sol</code>	413	<code>e0b81c82d82798fd</code>	Per-campaign USDC escrow (EIP-1167 clone): donations, finalisation, single or milestone payout, donor votes, refunds, Guardian freeze/resolve
<code>src/EmergencyPool.sol</code>	290	<code>d614b3d50987e1f5</code>	Sub-pools, gifts, inflows from campaigns, allocation proposals with checkpointed votes, Guardian resolve, reclaim
<code>src/PlatformConfig.sol</code>	98	<code>30ae85eea2f7b14f</code>	Parameters and roles (OpenZeppelin <code>AccessControl</code>); admin = timelock
<code>src/CampaignFactory.sol</code>	63	<code>d8589d3ef4724bfc</code>	Operator-only deterministic clone deployment; campaign registry
<code>src/IEmergencyPool.sol</code>	4	<code>2f15fb781947725d</code>	Interface used by campaigns to report inflows

Repository `DataVallis-CHERR-IO/web3-platform`, folder `packages/contracts`. Version 1.0: commit `a70d53d` (branch `dev`, 10 October 2026). Version 1.1: the TASK-023c batch (branch `feat/TASK-023c-contract-batch`, squash-merged into `dev`; the hashes above identify the reviewed files).

Toolchain: Solidity 0.8.24 (checked arithmetic), optimizer on with 200 runs, OpenZeppelin Contracts 5.7.0, Foundry (forge) and Slither 0.11.6.

Also read, for roles only: the deployment scripts `DeployPolygon.s.sol`, `DeployAmoy.s.sol` and `DeployCore.s.sol` (who receives which role; whether the deployer renounces).

Out of scope:

- the OpenZeppelin library itself;
- Circle's USDC contract;
- the off-chain web app, indexer and worker;
- gas optimisation.

The Amoy test deployment (`0x4d25...3A16` PlatformConfig, `0xd5Ca...5a00` CampaignFactory, `0xFa7F...9517` EmergencyPool) was **not** compared byte-for-byte with this commit: the block explorer is not reachable from the review environment. Bytecode verification is part of the mainnet checklist (§10).

METHOD

- 1 Manual review** of every function: who may call it, in which state, what it writes, which tokens it moves, and what happens on revert. The review also covers each state machine: the campaign's nine states and the allocation's seven states.
- 2 Static analysis** with Slither 0.11.6 (102 detectors). Every result was triaged (§6).
- 3 Test suite review and coverage.** The project's 254 tests were run, and line, statement, branch and function coverage was measured (§7).
- 4 Evidence tests.** `test/audit/ReviewFindings.t.sol` (6 tests) reproduces L-01, L-02, L-06 and I-01 as they behave today, and proves that the gas-griefing scenario in I-10 is not exploitable. For I-10 the test was first shown to fail against a deliberately broken contract.
- 5 Property checklist** (§8): access control, reentrancy, arithmetic, rounding, conservation, initialisation, front-running, denial of service, and the effects of external tokens.

03 SYSTEM OVERVIEW

CONTRACTS AND MONEY FLOW

MONEY	FROM	TO	WHEN
Donation	Donor	Campaign escrow (one contract per campaign)	While LIVE, before the deadline
Payout	Campaign	Fixed beneficiary	Success ($\geq$ threshold of target, default 10 %): one payment after the release delay, or 3 milestone tranches after donor votes
Platform fee ($\leq$ 5 %)	Campaign	Treasury	With the single payment or the first tranche
Refund	Campaign	Donor	FAILED (full) or REJECTED (pro-rata of what is left)
Pool settlement / sweep	Campaign	EmergencyPool	Donor's choice, or unclaimed after the refund window
Gift	Giver	EmergencyPool sub-pool	Any time
Allocation	EmergencyPool	Live factory campaign	Operator proposes; contributors vote, or the Guardian decides
Reclaim	Failed campaign	EmergencyPool (funding sub-pool)	After failure or rejection

ROLES AND WHAT THEY CAN DO

ROLE	HOLDER ON MAINNET (EPOCH POLYGON S. SOL)	CAN	CANNOT
DEFAULT_ADMIN_ROLE (PlatformConfig)	TimelockController, 48h, proposer and executor = Timelock Safe	Change parameters within their bounds; set <code>treasury</code> and <code>emergencyPool</code> ; grant and revoke roles	Touch a running campaign's snapshotted rules; move escrowed USDC directly
OPERATOR_ROLE	Operator Safe	Create campaigns (beneficiary, target, deadline 1–90 days); set the payout mode once after success; create sub-pools; propose pool allocations to live factory campaigns	Change a beneficiary; release money early; skip a donor vote

SYSTEM OVERVIEW

GUARDIAN_ROLE	Guardian Safe (a different one since v1.1, M-01)	Freeze a campaign; resolve a frozen or under-review campaign (next tranche or rejection); resolve an allocation under review	Send campaign money anywhere other than the beneficiary or back to donors
Anyone	–	Donate; finalise after the deadline; count votes after the window; settle a donor to the pool; sweep after the refund window; reclaim pool money from a failed campaign	–

Section 9 discusses what follows from this table.

04 SEVERITY DEFINITIONS

SEVERITY	MEANING
Critical	Direct loss or theft of user funds, or a permanent freeze, reachable by an unprivileged account.
High	Loss or freeze of funds under realistic conditions, or a privileged role able to take funds beyond its documented power.
Medium	A trust or governance weakness, or a condition that can misdirect funds or block a core flow, that needs a mitigation before mainnet.
Low	An edge case or design choice with limited impact that deserves a decision, a test or documentation.
Informational	Style, clarity, defensive-coding or documentation points; no impact on funds.

05 FINDINGS

ID	TITLE	SEVERITY	STATUS
M-01	One Safe holds Operator, Guardian and timelock proposer/executor	Medium	Fixed in the deployment script (v1.1); Safes to be created
M-02	Live-read <code>emergencyPool</code> and <code>treasury</code> let the timelock redirect future flows of running campaigns	Medium	Open — monitoring, documented
L-01	Sub-pool money in a failed campaign returns to the general pool if the campaign is swept before it is reclaimed	Low	Fixed (v1.1)
L-02	A campaign stays bound to the sub-pool of its first proposal, even if that proposal was rejected	Low	Fixed (v1.1)
L-03	The whole fee is paid with the first milestone and is not refunded after a rejection	Low	Fixed (v1.1, fee per tranche)
L-04	A USDC-blacklisted beneficiary or treasury blocks payouts until the Guardian steps in	Low	Acknowledged — runbook
L-05	A donor's last donation sets the failure preference for all of their money in the campaign	Low	Acknowledged — documented
L-06	Freezing a live campaign does not extend its deadline	Low	Fixed (v1.1)
I-01	Unknown allocation ids read as VOTING; <code>closeAllocation</code> on one reverts with an arithmetic panic	Info	Fixed (v1.1)
I-02	Slither <code>reentrancy-no-eth</code> / <code>reentrancy-benign</code> : false positives	Info	Triaged
I-03	Slither <code>unused-return</code> on <code>Checkpoints.push</code> : false positive	Info	Triaged
I-04	Timestamp comparisons	Info	Accepted
I-05	Parameter naming (<code>_x</code>)	Info	Accepted
I-06	USDC sent directly to the pool, or to a completed campaign, cannot be recovered	Info	Acknowledged
I-07	<code>minDonation</code> is read live and has no upper bound	Info	Fixed (v1.1, upper bound)
I-08	Pool voting weight is lifetime contribution; the target campaign's people may vote	Info	Acknowledged — product

I-09	<code>receiveFromCampaign</code> trusts the amount reported by factory campaigns	Info	Acknowledged
I-10	Gas griefing of the delivery <code>try/catch</code> — analysed, not exploitable	Info	Verified by test

M-01 — ONE SAFE HOLDS OPERATOR, GUARDIAN AND TIMELOCK PROPOSER/EXECUTOR

Location: `script/DeployPolygon.s.sol` (lines 55–67).

Description. On mainnet the same Safe receives `OPERATOR_ROLE` and `GUARDIAN_ROLE`, and it is also the only proposer and executor of the 48-hour timelock that holds `DEFAULT_ADMIN_ROLE`. Every power in §3 therefore sits behind one signer set.

Impact. The contracts stop any role from withdrawing escrowed money directly. A compromised or colluding Safe could still move **Emergency Pool** money in five steps:

- 1 create a campaign with a beneficiary it controls (Operator);
- 2 propose a pool allocation to it (Operator);
- 3 wait for low turnout, which is common in pool votes;
- 4 approve the allocation as Guardian;
- 5 set the payout mode and wait for release.

The checks against this are weaker than they look. Contributors can vote the allocation down, and the beneficiary is fixed at creation. But a low-turnout vote ends in Guardian review by design (ADR-045), and the Guardian is the same Safe. Campaign donors' money is not exposed in the same way: it only goes to the beneficiary that the donors chose to fund.

Recommendation.

- Use separate Safes with different signer sets for Operator (day-to-day publishing), Guardian (disputes) and the timelock proposer (configuration).
- Publish the signers.
- Give the Guardian Safe a higher threshold.
- Optionally, require a minimum turnout for Guardian *approval* of pool allocations, or a delay between `NEE_DS_REVIEW` and `RESOLVED_PASS`. This needs a contract change.

Resolution (v1.1). `DeployPolygon.s.sol` now takes `OPERATOR_SAFE`, `GUARDIAN_SAFE` and `TIMELOCK_SAFE`. The timelock's only proposer and executor is the Timelock Safe. Each address must be a contract, and the script reverts with `SafesMustDiffer` if two are the same. `test_DeployPolygon_guards` runs the real script and checks that each Safe holds only its own role and the deployer keeps none. **Still to do (Data Vallis):** create the three Safes, choose the signers, and give the Guardian Safe a higher threshold. The optional contract-level turnout rule for Guardian approvals was not added.

M-02 — LIVE-READ EMERGENCYPOOL AND TREASURY LET THE TIMELOCK REDIRECT FUTURE FLOWS OF RUNNING CAMPAIGNS

Location: `Campaign.sol` — `settleToPool`, `sweepUnclaimed`, `donateFromPool` (lines 223–246, 534–578, 595–598) and `release` / `_releaseNextTranche` (`config.treasury()`); `PlatformConfig.setEmergencyPool`, `setTreasury`.

Description. Campaigns read `config.emergencyPool()` and `config.treasury()` when they transfer, not at creation. `setEmergencyPool` accepts any non-zero address. After the 48-hour timelock, the admin can therefore send all *future* pool-bound money of every running campaign to a new address: donors’ “send to the pool” choices and the unclaimed sweeps. The same applies to future fees through `setTreasury`.

Impact. This misdirects funds that donors meant for the Emergency Pool. It cannot happen silently: the change is visible on chain for 48 hours, and Admin → Contracts shows scheduled changes. The live read is intentional, because it lets the pool be migrated without redeploying campaigns.

Recommendation.

- Run an off-chain watcher: an alert, plus a public notice, on every `CallScheduled` for `setEmergencyPool`, `setTreasury`, `grantRole` and `revokeRole`.
- Document in the owner guide that a pool migration is announced in advance.
- Before mainnet, consider restricting `setEmergencyPool` to a contract that implements `IEmergencyPool` and is bound to the same factory (an `extcodesize` check plus `factory()` equality).

Status (v1.1). Unchanged by design: the live read is what makes a pool migration possible without redeploying campaigns. The watcher is on the mainnet checklist (§10, item 5).

L-01 — SUB-POOL MONEY IN A FAILED CAMPAIGN RETURNS TO THE GENERAL POOL IF THE CAMPAIGN IS SWEEPED BEFORE IT IS RECLAIMED

Location: `EmergencyPool.reclaimFromCampaign` (270–287); `Campaign.sweepUnclaimed` (561–578).

Description. When pool money reaches a campaign that later fails or is rejected, `reclaimFromCampaign` returns it to the campaign’s funding sub-pool. Anyone may call it. If nobody does before the campaign’s refund window ends (180 days by default), `sweepUnclaimed` sends the campaign’s whole balance to the pool with sub-pool 0, because the campaign does not know which sub-pool funded it. After that, `reclaimFromCampaign` reverts.

Evidence (v1.0): `test_L01_sweepSendsSubpoolMoneyToGeneralPool` showed 50 USDC from sub-pool 2 ending up in the general pool.

Impact. Money given for one theme (for example “Animals in danger”) can end up in the general pool. No money is lost.

Recommendation.

FINDINGS

- Process: the admin queue or the worker calls `reclaimFromCampaign` as soon as a campaign that received pool money is FAILED or REJECTED. The indexer knows these campaigns from `CampaignInfo` `low` and `Donated` with the pool as donor.
- Contract batch: `sweepUnclaimed` could first call `EmergencyPool.reclaimFromCampaign`, or report the pool's share separately.

Resolution (v1.1). `sweepUnclaimed` now first computes the Emergency Pool's own unreclaimed share of the campaign. That is its full donation for a FAILED campaign, or its pro-rata part for a REJECTED one. The share goes to the pool with `donor = pool`. `EmergencyPool.receiveFromCampaign` recognises that case: it credits the campaign's funding sub-pool, gives nobody voting weight and emits `ReclaimedFromCampaign`. Only the rest is a plain sweep to pool 0, and `settled[pool]` stops any double counting. Evidence:

- `test_L01_sweepReturnsPoolMoneyToFundingSubpool` (FAILED);
- `test_L01_sweepOfRejectedCampaignReturnsProRataShare` (REJECTED);
- `test_L01_reclaimThenSweepDoesNotDoubleCount`.

The tests fail when the fix is undone: `the funding sub-pool gets its 50 USDC back: 4500000000 != 500000000`.

L-02 — A CAMPAIGN STAYS BOUND TO THE SUB-POOL OF ITS FIRST PROPOSAL, EVEN IF THAT PROPOSAL WAS REJECTED

Location: `EmergencyPool.proposeAllocation` (165–170).

Description. `hasFundingPool[campaign]` and `fundingPool[campaign]` are set when the first proposal is made and are never cleared, even if that allocation is rejected, returned or fails delivery. Every later proposal to the campaign from another sub-pool reverts with `PoolIdMismatch`.

Evidence (v1.0): `test_L02_fundingPoolBindingSurvivesRejectedProposal`.

Impact. One rejected proposal stops the Operator from funding a campaign from any other sub-pool, including the general pool. The rule exists so that `reclaimFromCampaign` can credit one sub-pool. Money only reaches a campaign through PASSED or RESOLVED_PASS allocations, so the rule only has to hold for those.

Recommendation (contract batch): bind the campaign only when money is actually delivered (in the `try` branch of the delivery helpers), and keep the mismatch check against that delivered binding.

Resolution (v1.1). `hasFundingPool` / `fundingPool` are set only when money is delivered (PASSED or RESOLVED_PASS). While proposals are open, `reservedPool` and `openAllocations` keep a second sub-pool from proposing to the same campaign. Every final state (PASSED, REJECTED, RESOLVED_PASS, RESOLVED_REJECT, DELIVERY_FAILED) closes the reservation. NEEDS_REVIEW keeps it. The indexer now sets `funding_pool_id` on delivery as well. Evidence: `test_L02_rejectedProposalDoesNotBindCampaign`, `test_L02_openProposalReservesAndDeliveryBinds`.

L-03 — THE WHOLE FEE IS PAID WITH THE FIRST MILESTONE AND IS NOT REFUNDED AFTER A REJECTION

Location: `Campaign._releaseNextTranche` (lines 337–341) and `_reject` (497–502).

Description. In milestone mode the first tranche pays the **whole** platform fee, calculated on the total raised. If donors later reject the campaign, `rejectedRemainder = totalRaised - released - feePaid`, so donors get back their share of the unpaid tranches *minus* the fee on those tranches.

Impact. At the default 1 % fee and a rejection after the first tranche, donors lose about 0.67 % of their donation to the fee on money that was never paid out. This is already listed as a known limitation in the technical reference (02 §10, item 12).

Recommendation: product decision. Either charge the fee per tranche (fee/3 with each), or keep the current rule and state it on the donate panel and in the terms.

Resolution (v1.1, Data Vallis decision). Each tranche now carries a third of the fee; the third tranche absorbs the rounding dust of both the net amount and the fee. A rejection after the first or second tranche returns the unpaid fee to donors through `rejectedRemainder`. SINGLE payouts are unchanged. The fuzz test `testFuzz_trancheMath` checks that the three tranches carry exactly the whole fee, and the campaign invariants still hold. Evidence: `test_L03_rejectionRefundsUnpaidFee`. The terms and the campaign page say so.

L-04 — A USDC-BLACKLISTED BENEFICIARY OR TREASURY BLOCKS PAYOUTS UNTIL THE GUARDIAN STEPS IN

Location: `Campaign.release`, `_releaseNextTranche`, `closeVote`.

Description. USDC can blacklist addresses. If the beneficiary or the treasury is blacklisted, every payout reverts: `release`, the tranche paid inside `closeVote`, and Guardian `resolve(true)`. The campaign stays in SUCCEEDED, PAYING or VOTING. The way out is Guardian `freeze()` followed by `resolve(false)`; donors then claim pro-rata refunds. A blacklisted treasury blocks every campaign's fee transfer until the timelock sets a new treasury (48 hours).

Recommendation: keep the escape path, and add it to the mainnet runbook. Screen beneficiary addresses against the USDC blacklist at publication; this is not built yet (ADR-054 only covers sanctioned countries). Use a treasury Safe.

L-05 — A DONOR'S LAST DONATION SETS THE FAILURE PREFERENCE FOR ALL OF THEIR MONEY IN THE CAMPAIGN

Location: `Campaign.donate` (lines 207–208).

Description. Each `donate` overwrites `preference` and `donorSubPoolId` for the donor's whole balance in the campaign. A donor who gave twice with different choices gets the last one for both gifts. `setPreference` can still change it while the campaign is live. This is documented in the technical reference (02 §10, item 6).

Recommendation: keep it, and show the current preference on the donate panel when a donor gives again.

L-06 — FREEZING A LIVE CAMPAIGN DOES NOT EXTEND ITS DEADLINE

Location: `Campaign.freeze / resolve` (449–494).

Description. When a frozen campaign is resolved back to VOTING, it gets the frozen time back (`voteEnd + = frozenDuration`). A campaign frozen while LIVE does not: its deadline keeps running. If it is unfrozen after the deadline, it can only be finalised; no further donations are possible.

Evidence (v1.0): `test_L06_freezeDoesNotExtendLiveDeadline`.

Recommendation (contract batch): extend `deadline` by the frozen duration when restoring LIVE. Otherwise, document that a freeze during fundraising costs the campaign that time.

Resolution (v1.1). `resolve(true)` on a campaign frozen while LIVE adds the frozen duration to `deadline`, just as it already did for `voteEnd`. No cap is applied: the extension equals the time the Guardian kept the campaign frozen. Evidence: `test_L06_unfreezeExtendsLiveDeadline`.

I-01 — UNKNOWN ALLOCATION IDS READ AS VOTING; `CLOSEALLOCATION` ON ONE REVERTS WITH AN ARITHMETIC PANIC

Location: `EmergencyPool.getAllocation`, `closeAllocation` (218–223).

Description. `AllocationState.VOTING` is enum value 0, so `getAllocation(id)` for an id that was never proposed returns a zeroed struct in state VOTING. `closeAllocation(id)` then reaches `a.proposalBlock - 1` with `proposalBlock == 0` and reverts with `Panic 0x11` instead of a named error. No state can change.

Evidence (v1.0): `test_I01_unknownAllocationReadsAsVotingAndPanicsOnClose`.

Recommendation (contract batch): add `error AllocationDoesNotExist()` and check `id < allocationCount`, or make the first enum value `NONE`.

Resolution (v1.1). `getAllocation`, `voteAllocation`, `closeAllocation` and `resolveAllocation` revert with `AllocationDoesNotExist` for `id >= allocationCount`. Evidence: `test_I01_unknownAllocationHasNamedError`.

I-02 — SLITHER REENTRANCY-NO-ETH AND REENTRANCY-BENIGN : FALSE POSITIVES

`_deliverAllocation` and `_deliverAllocationResolve` write state after calling `Campaign.donateFromPool`. Their only callers, `closeAllocation` and `resolveAllocation`, are `nonReentrant`. The callee is factory-deployed `Campaign` code, which only performs a USDC `transferFrom`; USDC has no callbacks.

`reclaimFromCampaign` is `nonReentrant` and measures the balance change. The existing test `MaliciousForReentrancy` (`EmergencyPool.t.sol`) shows that a re-entering campaign is refused. No change is needed.

I-03 — SLITHER `UNUSED-RETURN ON CHECKPOINTS.PUSH` : FALSE POSITIVE

`Checkpoints.Trace256.push` returns the previous and the new value, and the code does not need them. No change is needed.

I-04 — TIMESTAMP COMPARISONS

Eleven comparisons with `block.timestamp` (deadlines, vote windows, delays). Polygon block producers can shift timestamps by seconds; every window here is at least one hour. Accepted.

I-05 — PARAMETER NAMING

Sixteen setter and initialiser parameters use a leading underscore (`_feeBps`), which Slither reports against `mixedCase`. Style only; accepted.

I-06 — USDC SENT DIRECTLY CANNOT BE RECOVERED

USDC transferred straight to the `EmergencyPool` is not counted in any `poolBalance` and has no rescue path. For a campaign it depends on the state: in `FAILED` or `REJECTED` it is swept to the pool, but in `COMPLETED` it stays locked. Documented (02 §10, item 8). A rescue function would add an owner power, so none is recommended.

I-07 — `MINDONATION` IS READ LIVE AND HAS NO UPPER BOUND

`Campaign.donate` and `EmergencyPool.donate` read `config.minDonation()` live, and the setter only refuses 0. A very high value would block new donations everywhere, after 48 hours. Recommendation: cap it in the setter (for example at 1,000 USDC) in the contract batch. **Resolution (v1.1):** `MAX_MIN_DONATION = 1,000 USDC`, so the setter reverts with `MinDonationTooHigh`. Admin → Contracts validates the same bound. Evidence: `test_I07_minDonationHasUpperBound`.

I-08 — POOL VOTING WEIGHT IS LIFETIME CONTRIBUTION; THE TARGET CAMPAIGN'S PEOPLE MAY VOTE

The weight is everything an address gave to a sub-pool before the proposal block. It does not go down when the pool spends money, and nothing stops the beneficiary of the proposed campaign from voting if they contributed. This is a product rule (ADR-045 uses money-weighted votes). It is worth stating on the `Emergency Pool` page.

I-09 — RECEIVEFROMCAMPAIGN TRUSTS THE REPORTED AMOUNT

The pool credits the `amount` that a factory campaign reports, without measuring its own balance. This is safe while every factory campaign runs the fixed `Campaign` code; there is one implementation and no upgrades. Documented (02 §10, item 11).

I-10 — GAS GRIEFING OF THE DELIVERY TRY/CATCH : ANALYSED, NOT EXPLOITABLE

The worry is that a caller of `closeAllocation` picks a gas limit at which `donateFromPool` runs out of gas inside the `try`, while the remaining 1/64 is still enough for the `catch` branch. That would turn an approved allocation into `DELIVERY_FAILED`. With the 63/64 rule, the attack needs `catch` cost $\times 64 <$ inner call cost; here the `catch` costs about as much as the call itself. `test_I10_limitedGasCannotForceDeliveryFailed` sweeps gas limits from 30,000 to 400,000 in steps of 2,500. It asserts that every successful close delivered (`PASSED`).

The test was shown to fail when the contract was deliberately broken to cap the inner call's gas: `a gas-limited close must not end in DELIVERY_FAILED: 6 != 1`. The test uses a mock USDC. Real USDC costs more gas inside the call, which only widens the margin.

06 STATIC ANALYSIS (SLITHER)

Command: `slither . --config-file slither.config.json` (filters `lib/`, `test/` and `script/`), Slither 0.11.6, 25 contracts, 102 detectors. Since TASK-023a CI runs it on every pull request that touches the contracts, with `--fail-high`.

IMPACT	DETECTOR	RESULTS	TRIAGE
High	–	0	–
Medium	<code>reentrancy-no-eth</code>	2	False positive (I-02)
Medium	<code>unused-return</code>	2	False positive (I-03)
Low	<code>reentrancy-benign</code>	3	False positive (I-02)
Low	<code>timestamp</code>	11	Accepted (I-04)
Informational	<code>naming-convention</code>	16	Style (I-05)
Total		34	

These are the same figures as the manual run in TASK-004 (High 0, Medium 4, Low 14, Informational 16), and the same again on the v1.1 code. While the v1.1 batch was being written, Slither flagged one new `uninitialized-local` (Medium) on `poolShare` in `sweepUnclaimed`. The variable is now initialised explicitly, and the count is back to 34. CI fails on any High result. A new Medium or Low result has to be triaged in this report before the next version.

07 TESTING AND COVERAGE

TEST SUITES (FOUNDRY, **FORGE TEST**)

SUITE	TESTS	KIND
Campaign.t.sol	105	Unit: every function, every state, every revert
PlatformConfig.t.sol	49	Unit: setters, bounds, roles
EmergencyPool.t.sol	43	Unit, including re-entrancy and delivery failure
Deploy.t.sol	18	The deployment scripts: role layout, renounce, guards; the real DeployPolygon run with three Safes
CampaignFactory.t.sol	15	Unit
CampaignFuzz.t.sol	6	Fuzz, 1,000 runs each: donation clipping, fee maths, success threshold, refund sums, tranche maths, pro-rata remainder
invariant/CampaignInvariant.t.sol	10	Invariants, 256 runs × 128 calls
invariant/PoolInvariant.t.sol	4	Invariants, 256 runs × 128 calls
Placeholder.t.sol	1	–
audit/ReviewFindings.t.sol	10	Evidence for the fixes of L-01 (3), L-02 (2), L-03, L-06, I-01, I-07, and the I-10 gas sweep
Total	261	All pass

Campaign invariants:

- the balance equals what the campaign owes;
- raised never exceeds the target;
- the balance is zero after COMPLETED and after a sweep;
- single and milestone payouts are exact;
- the ghost accounting matches the chain.

Pool invariants:

- balance conservation (USDC held = free balances + reserved allocations);
- no negative pool balance.

COVERAGE (FORGE COVERAGE , SOURCE FILES ONLY)

FILE	LINES	STATEMENTS	BRANCHES	FUNCTIONS
Campaign.sol	97.61 % (245/251)	96.28 % (311/323)	91.76 % (78/85)	100 % (21/21)
CampaignFactory.sol	100 % (22/22)	100 % (31/31)	100 % (7/7)	100 % (4/4)
EmergencyPool.sol	98.75 % (158/160)	95.90 % (187/195)	83.33 % (40/48)	100 % (17/17)
PlatformConfig.sol	100 % (45/45)	100 % (58/58)	100 % (12/12)	100 % (11/11)
Total	98.33 %	96.71 %	90.13 %	100 %

Coverage was measured with `--ir-minimum`. Foundry warns that this mode can map source lines inaccurately, and the line report confirms it: it shows zero hits on lines that dedicated tests exercise, for example the release and reject branches of `closeVote` (`Campaign.t.sol` lines 916–1004) and of `resolve` (`test_resolve_approveNeedsReview_releasesTrancheAtomically` , `test_resolve_reject_fromNeedsReview`). The figures above are therefore a **lower bound**. Unmapped branches that are genuinely worth a test before the external audit:

- the constructor's zero-address checks in `EmergencyPool` ;
- `reclaimFromCampaign` on a live campaign;
- the partial-return branch of `_deliverAllocationResolve` .

08 PROPERTIES CHECKED

PROPERTY	RESULT
Every state-changing privileged function checks its role through <code>PlatformConfig.hasRole</code>	✓ (Appendix A)
No function sends escrowed USDC to a caller-chosen address	✓ Beneficiary fixed at creation; refunds go to <code>msg.sender</code> for its own donated ; pool flows go to factory campaigns or <code>config.emergencyPool()</code> (M-02)
Re-entrancy	✓ <code>nonReentrant</code> on every token-moving function; CEI order; USDC has no hooks (I-02)
Arithmetic	✓ Solidity 0.8 checked arithmetic; fee $\leq 5\%$; multiplications before divisions; no division by zero (each denominator is checked or positive by state)
Rounding	✓ Pro-rata refunds round down; dust stays and is swept; the third tranche absorbs tranche dust (fuzz <code>testFuzz_trancheMath</code> , <code>testFuzz_proRataRemainder</code>)
Conservation of funds	✓ Campaign and pool invariants (§7)
Clone initialisation	✓ The implementation calls <code>_disableInitializers()</code> ; the factory clones and initialises in one transaction; OZ 5.7 <code>ReentrancyGuard</code> uses a storage slot and treats the clone's initial 0 as “not entered”
Front-running	✓ Only the Operator can create campaigns, so deterministic addresses cannot be squatted. Pool vote weight comes from the block before the proposal; a front-run gift in the same block gets no weight, and any gift is irreversible
Denial of service	✓ No unbounded loops; pull payments; a failed delivery returns money instead of reverting (DELIVERY_FAILED). Blacklisted USDC addresses: L-04
Snapshot integrity	✓ Running campaigns and open allocations keep their own parameters; only <code>treasury</code> , <code>emergencyPool</code> and <code>minDonation</code> are read live (M-02, I-07)
Events	✓ Every state change emits an event. A few state transitions have no dedicated state event and are derived from <code>TrancheReleased</code> / <code>EvidenceSubmitted</code> (02 §10, item 13)
Upgradeability / self-destruct / <code>delegatecall</code>	✓ None (clones only delegate to the fixed implementation)

09 CENTRALISATION AND TRUST ASSUMPTIONS

CHERR.IO is a curated platform: organisations pass KYB, campaigns are approved and published by the Operator, and disputes are settled by the Guardian. The trust model is therefore explicit:

- **Donors trust the Operator** to publish only verified campaigns with the right beneficiary address. The contracts make sure the money then goes only to that address, or back to donors.
- **Donors in milestone campaigns** keep control through their votes. Silence never counts as consent: low turnout goes to the Guardian (ADR-045).
- **The Guardian** can freeze any active campaign and reject it, which refunds donors. It can approve the next tranche of a campaign under review. It decides pool allocations under review. It cannot send money anywhere else.
- **Pool contributors trust the Operator and the Guardian** more than campaign donors do (M-01; since v1.1 these are two different Safes): allocations go to campaigns the Operator chose, and low turnout leaves the decision to the Guardian.
- **The timelock admin** can change parameters within hard-coded bounds and redirect future pool and fee flows (M-02). Everything it does is public for 48 hours first.

Mainnet governance recommendations:

- separate Safes (M-01; enforced by the deployment script since v1.1);
- a timelock watcher with public announcements (M-02);
- published signer sets;
- a documented incident procedure for freeze and resolve.

10 MAINNET READINESS CHECKLIST

#	ITEM	OWNER	STATUS
1	External audit by a recognised firm, with this report and <code>test/audit</code> as input	Data Vallis (budget)	Open
2	Contract batch: L-01 (sweep → funding sub-pool), L-02 (bind on delivery), L-06 (extend deadline on unfreeze), I-01 (named error), I-07 (cap <code>minDonation</code>), with tests	David → CTO	Done (v1.1, TASK-023c); Amoy redeploy by David open
3	L-03: fee per tranche	David	Done (v1.1, ADR-061)
4	Separate Operator, Guardian and timelock Safes; publish the signers (M-01)	David	Script done (v1.1); creating the Safes open
5	Timelock watcher and public notice on <code>CallScheduled</code> (M-02)	CTO (worker job)	Open
6	Prompt <code>reclaimFromCampaign</code> for failed campaigns that hold pool money (L-01)	CTO	No longer needed for correctness (v1.1 sweep returns the money to its sub-pool); optional for speed
7	Screen beneficiaries against the USDC blacklist; runbook entry for freeze → reject (L-04)	CTO	Open
8	Verify the deployed bytecode against the audited commit on Polygonscan	CTO + David	Open
9	Slither in CI (fails on High)	CTO	Done (TASK-023a)
10	Gas Manager allow-list for the donor and voter calls only (<code>donate</code> , <code>approve</code> , <code>vote</code> , <code>claimRefund</code> , <code>settleToPool</code> , <code>EmergencyPool.donate</code> , <code>voteAllocation</code> , <code>closeAllocation</code>)	David (Alchemy)	Open
11	Indexer finality on Polygon (200 blocks) reviewed	CTO	Open
12	Bug bounty after launch	Data Vallis	Planned

11 REPRODUCING THE RESULTS

```
cd packages/contracts
git submodule update --init --depth 1 lib/forgo-std lib/openzeppelin-contracts
forge test                                # 261 tests
forge test --match-path test/audit/ReviewFindings.t.sol -vv
forge coverage --no-match-coverage "(script|test)" --report summary --ir-minimum
pip install slither-analyzer==0.11.6
slither . --config-file slither.config.json          # 34 results, 0 High
slither . --config-file slither.config.json --fail-high
```

APPENDIX A — ACCESS CONTROL BY FUNCTION

CONTRACT · FUNCTION	CALLER	STATE REQUIRED
Factory · <code>createCampaign</code>	Operator	– (deadline 1–90 days, target $\geq$ 100 USDC, unique id)
Campaign · <code>initialize</code>	Factory (once, same transaction as the clone)	–
Campaign · <code>donate</code>	Anyone	LIVE, before the deadline, $\geq$ <code>minDonation</code>
Campaign · <code>donateFromPool</code>	<code>config.emergencyPool()</code>	LIVE, before the deadline
Campaign · <code>setPreference</code>	A donor	LIVE
Campaign · <code>finalize</code>	Anyone	LIVE, after the deadline
Campaign · <code>setPayoutMode</code>	Operator	SUCCEEDED, once; SINGLE refused for individuals
Campaign · <code>release</code>	Anyone	SUCCEEDED, payout mode set; SINGLE after the release delay
Campaign · <code>submitEvidence</code>	Beneficiary	PAYING
Campaign · <code>vote</code>	A donor, once per round	VOTING, before <code>voteEnd</code>
Campaign · <code>closeVote</code>	Anyone	VOTING, after <code>voteEnd</code>
Campaign · <code>freeze</code>	Guardian	LIVE, SUCCEEDED, PAYING, VOTING, NEEDS_REVIEW
Campaign · <code>resolve</code>	Guardian	FROZEN or NEEDS_REVIEW
Campaign · <code>claimRefund</code>	A REFUND-preference donor, once	FAILED or REJECTED, not swept
Campaign · <code>settleToPool</code>	Anyone, for a POOL-preference donor, once	FAILED or REJECTED, not swept
Campaign · <code>sweepUnclaimed</code>	Anyone, once	FAILED or REJECTED, after the refund window
Pool · <code>createSubPool</code>	Operator	Sub-pool does not exist yet
Pool · <code>donate</code>	Anyone	$\geq$ <code>minDonation</code> (unknown sub-pool $\rightarrow$ 0)
Pool · <code>receiveFromCampaign</code>	A factory campaign	–

Pool · proposeAllocation	Operator	Sub-pool exists and has the balance; campaign is a LIVE factory campaign whose deadline is after the vote end; the same sub-pool as an earlier delivery or an open proposal to that campaign, if any (v1.1)
Pool · voteAllocation	A contributor with weight, once	Allocation exists (v1.1), VOTING, before voteEnd
Pool · closeAllocation	Anyone	VOTING, after voteEnd
Pool · resolveAllocation	Guardian	NEEDS_REVIEW
Pool · reclaimFromCampaign	Anyone	Campaign FAILED or REJECTED, pool is a donor, not swept
Config · setters, grantRole , revokeRole	DEFAULT_ADMIN_ROLE (timelock)	Within the hard-coded bounds

APPENDIX B — DOCUMENT HISTORY

VERSION	DATE	CHANGE
1.0	2026-10-10	First version (TASK-023a): scope commit <code>a70d53d</code> , 18 findings (0 C, 0 H, 2 M, 6 L, 10 I), Slither in CI, evidence tests.
1.1	2026-10-10	Re-review after the TASK-023c batch (ADR-061, approved by Data Vallis): M-01 fixed in <code>DeployPolygon.s.sol</code> (three different Safes); L-01, L-02, L-03 (fee per tranche), L-06, I-01, I-07 fixed in code, each with a test; 261 tests; Slither 34 results, 0 High. Amoy redeploy pending.



CHERR.IO

Internal pre-audit review of the CHERR.IO contracts